ML-GS: An Implementation of Gradual Security Typing with References

Eren Yenigül

June 2026

1 Scope and Motivation

This project implements ML-GS, the calculus introduced by Fennell and Thiemann in *Gradual Security Typing with References* [1]. ML-GS extends a monomorphic ML core language with references and higher-order functions with gradual information-flow control (IFC). Security casts allow programmers to mix statically and dynamically checked regions in the same program, bridging the gap between the precision of static type systems and the flexibility of dynamic monitoring.

The motivation is practical. Static IFC type systems such as FlowCaml [3] are often too conservative for real programs. For example, a function whose safety depends on a runtime flag (e.g., `isPrivileged`) cannot be verified statically even when it is semantically safe. By inserting casts, programmers can confine dynamic checks to those regions and keep the rest statically verified.

The idea of combining gradual typing with security typing originates with Disney and Flanagan [2], who studied this combination for a pure lambda calculus. Fennell and Thiemann build on their approach and extend it to an ML core language with references, which introduces significant complications not present in the pure setting. Gradual typing itself was introduced by Siek and Taha [5], and the dynamic IFC technique used here, the no-sensitive-upgrade (NSU) policy, is due to Austin and Flanagan [4]. The underlying static type system is adapted from Pottier and Simonet's type system for CoreML [3].

The key technical challenge is handling references under gradual typing. Unlike pure-lambda-calculus settings, references allow flow-sensitivity attacks, require invariant subtyping, and complicate the boundary between static and dynamic code. The paper addresses this with a distinction between *access types* and *current types* on heap cells, and with the NSU policy for dynamic IFC.

2 Approach and Main Results

The implementation is a Scala interpreter that covers the full ML-GS calculus from the paper. It consists of three main components.

Parser. A combinator parser that reads ML-GS source programs. The surface syntax extends the paper's formal notation with several conveniences: named functions (`fn`), multi-argument currying, sequencing (`;`), `let` bindings, and a simplified cast syntax (`e as t`) that infers the source type rather than requiring it to be written explicitly. Security level annotations are written as `[L]`, `[H]`, and `[?]`.

Type checker. Implements the typing rule $pc; \Gamma; M \vdash e : t$ (Figure 2 of the paper), including subtyping ($\prec$), compatibility ($\sim$), and the typing rules for casts (T-Cast), protection (T-Protect), guard casts (T-GuardCast), function guard casts (T-FunCast), and pointer casts (T-RefCast). The type checker enforces the program counter constraint on write effects and reports source-location errors.

Interpreter. Implements the small-step reduction relation $e/\mu \xrightarrow{PC} r$ from Figures 5, 6, and 9 of the paper, covering the following rules.

- Lambda application with protect wrapping (R-App, R-Protect).
- Heap allocation, dereference, and assignment with NSU enforcement (R-New, R-Deref, R-Asgn, R-Asgn-NSU-Fail).
- Cast reduction with propagation into sub-values (R-Cast-Sub, R-Cast-To-Dyn, cast propagation of Figure 7).
- Guard casts that adjust program counter constraints through assignments and function bodies (R-GuardCast-New, R-GuardCast-Asgn, R-GuardCast-App, R-FunCast, R-RefCast).
- Blame propagation: failures carry blame labels that identify the responsible cast.

A REPL is also provided for interactive exploration, preserving variable bindings and heap state across evaluations.

Examples

The `examples/` directory contains programs that exercise different aspects of the calculus. The two most illustrative are the `addPrivileged` variants from Section II of the paper.

False positive (`examples/facebook_fail.mlgs`). With fully static types, `addPrivileged` must be typed to cover both the `true` and `false` branches. The body contains `report := !infoH`, which writes a value of type `int[H]` into a `ref[L]<int[L]>`. The type checker rejects this with a type mismatch, even though the `false` branch never executes the write. This is the over-conservatism gradual typing is designed to address.

Gradual fix (`examples/facebook_dyn.mlgs`). Giving `addPrivileged` a fully dynamic signature and casting `sendToFacebook` at the call site resolves the static rejection. Security levels are checked at runtime instead.

```
// examples/facebook_dyn.mlgs
let w = sendToFacebook as (ref[?]<int[?]> -[?]-> unit[?])[L];
addPrivileged(false, w, report);
```

At runtime, no secret value flows to `sendToFacebook` and the program completes without a blame exception. Passing `true` instead would trigger a blame error when the secret write is attempted under a dynamic check.

The remaining examples cover NSU enforcement (`4.mlgs`), successful NSU under a low pc (`3.mlgs`), and high-security function values (`secret_func.mlgs`). Each is described in the repository README.

3 Evaluation

The implementation is evaluated via a test suite of 20 unit tests in `ProgramTests.scala`, covering the following scenarios.

- Basic values, let bindings, and function application.
- Heap allocation, dereference, and assignment.
- Security level propagation through pointer dereference.
- Cast acceptance: dynamic upcasts, H-to-? and L-to-? casts.
- Cast rejection: downcast of an H value to L raises a blame error at runtime.
- Static type errors: allocating an L reference under an H program counter.
- NSU violation: assigning to a public cell under a secret program counter.
- Higher-order functions and guard casts.

All 20 tests pass. All programs in the `examples/` directory also execute correctly, including the NSU, cast, and `addPrivileged` scenarios described above.

4 Limitations

- **Two-level lattice only.** The implementation uses the L/H lattice from the paper. The paper notes results generalize to arbitrary discrete lattices, but this is not implemented.
- **No polymorphism.** The calculus and implementation are monomorphic, as in the paper.
- **Termination insensitivity.** The system proves termination-insensitive non-interference (TINI). Timing and termination channels are not addressed.
- **Blame precision.** As discussed in Section VIII of the paper, blame labels can be imprecise because labels are joined unconditionally in R-Deref. The extended pointer syntax that would recover precision (also described in Section VIII) is not implemented.
- **No inference.** Cast placement is entirely manual. The paper lists automatic cast inference as future work.

5 Experience

The most interesting part of the project was working through the reference cast mechanism. The distinction between the access type of a pointer and the current type of a heap cell is subtle. A cast changes only the pointer's access type, not the stored value, and the interpreter must insert a reconciling cast at each dereference. Getting this right required reading the typing and reduction rules together several times.

One concrete misconception was the timing of blame errors on references. My initial intuition was that a blame error should fire at write time, when a secret value is stored into a cell. However, the implementation kept raising errors at dereference instead, which I first assumed was a bug. Reading the semantics more carefully revealed that this is intentional. The rule R-Deref is where the access type and the current type of the heap cell are reconciled, and a mismatch is only observable when the value is read out. Writes are permitted even when they change the cell's type annotation, because a static alias of the same pointer may still see the old type. The blame error surfaces at the first dereference that cannot reconcile the two types, not at the write that caused the divergence.

Another challenge was implementing guard cast propagation. When a GuardCast node wraps an `Assign`, the interpreter must decompose the assignment by applying a PointerCast to the left-hand side and propagate the guard cast into both sub-expressions before evaluating either. Similarly, for a GuardCast wrapping an `Apply`, a FunctionGuardCast must be introduced on the function position while the guard cast is pushed into the argument. Getting this decomposition right required careful cross-referencing of the reduction rules (Figure 9) with the typing rules (Figure 8), because the guard cast must be present in the right position for type preservation to hold at each intermediate step. A mistake in the order of wrapping caused the pc constraint to be dropped or applied to the wrong sub-expression, producing either unsound behaviour or spurious type errors that were difficult to trace back to the relevant rule.

Designing the surface syntax was an interesting problem in itself. The paper's formal notation uses mathematical conventions that do not translate directly into ASCII. Security levels appear as superscripts ($\texttt{int}^{\texttt{L}}$, $\texttt{ref}^{\texttt{L}}\ \texttt{int}^{\texttt{H}}$), and the function arrow carries its pc annotation as a superscript ($t_1 \xrightarrow{pc} t_2$). The first decision was to move all annotations into square brackets, giving `int[L]`, `ref[L]<int[H]>`, and `-[pc]->`. Square brackets were chosen for level annotations because they are lightweight and visually distinct from the type structure; angle brackets were chosen for the content type of references because they mirror the common generic-type notation from languages like Java or C++, making `ref[L]<int[H]>` read naturally as "a level-L reference to a level-H integer." The function arrow `-[pc]->` was designed to echo the paper's annotated arrow while remaining unambiguous to parse: the leading hyphen and trailing `->` bracket the annotation symmetrically. These choices made the syntax feel consistent and close enough to the paper that programs could be written by reading the formal rules directly, without needing a separate translation step.

Integrating the surface-syntax extensions also required careful thought. Features like multi-argument currying, named functions, and the `as` cast shorthand are not in the paper's formal system, so there is no rule to follow directly. Each extension had to be reasoned about in terms of what it desugars to and whether that desugaring preserves the type and security invariants. For example, a multi-argument function `fn f(x, y) -> e` desugars into nested lambdas, each carrying its own value-level annotation. Deciding which level to assign to the intermediate lambdas, and verifying that the resulting type is compatible with what the type checker infers for the outermost call, required thinking carefully about how the paper's T-Abs and T-App rules compose. The `as` cast shorthand similarly needed a decision: rather than asking the programmer to supply both source and target types, the type checker infers the source type from the sub-expression, which is sound but means the cast node in the AST is partially unresolved until the type checker runs.

The main challenge overall was the sheer number of mutually dependent concepts. Security levels, type annotations, blame labels, the PC context, and the heap current/access type split all interact in almost every reduction rule. Building the interpreter rule by rule and validating each with a targeted test case kept the process manageable. The paper is well-structured, and having the formal rules as a direct blueprint made the Scala code relatively straightforward once the data model was settled.

References

[1] L. Fennell and P. Thiemann, "Gradual security typing with references," in *2013 IEEE 26th Computer Security Foundations Symposium*, New Orleans, LA, USA, 2013, pp. 224–239, doi: 10.1109/CSF.2013.22.

[2] T. Disney and C. Flanagan, "Gradual information flow typing," in *STOP 2011*, 2011.

[3] F. Pottier and V. Simonet, "Information flow inference for ML," *ACM TOPLAS*, vol. 25, no. 1, pp. 117–158, Jan. 2003.

[4] T. H. Austin and C. Flanagan, "Efficient purely-dynamic information flow analysis," in S. Chong and D. A. Naumann, Eds., *PLAS*, Dublin, Ireland, June 2009, pp. 113–124, ACM.

[5] J. G. Siek and W. Taha, "Gradual typing for functional languages," in *Scheme and Functional Programming Workshop*, Sept. 2006.